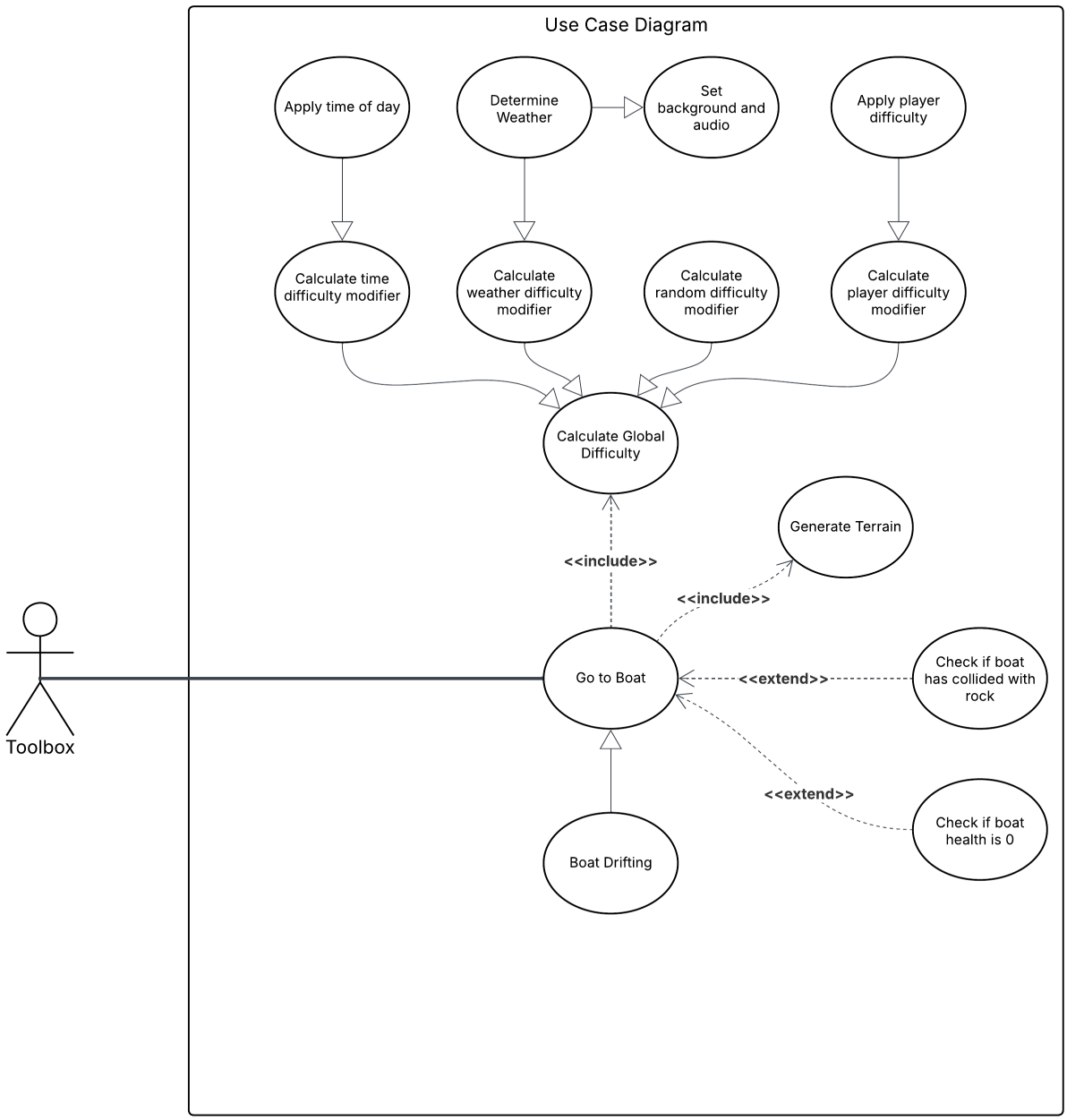
Name: **Johnathan Van Vliet** Mark _____/50

1. Brief introduction __/3

This Use Case Diagram introduces a scenario in which the Toolbox sends a signal to go to the boat for fishing. When the boat starts, a global difficulty is initially calculated. Then, terrain is generated. Then the boat drifts and the system checks if the boat has collided with a rock and if the boat health is 0.

2. Use case diagram with scenario __14

Use Case Diagrams



Scenarios

Name: Go to Boat

Summary: The player uses the Toolbox to go to the boat. When this happens, several things occur.

Actors: Toolbox

Preconditions: Boat must be made active using the toolbox.

Basic sequence:

- **Step 1:** Calculate the global difficulty
  - **Step 1a:** Determine weather based on RNG
  - **Step 1b:** Set background and audio to weather type
  - **Step 1c:** Calculate the weather difficulty modifier based on weather selection
  - **Step 1d:** Calculate the time difficulty modifier based on the time of day
  - **Step 1e:** Calculate the random difficulty modifier based on RNG
  - **Step 1f:** Calculate the player difficulty modifier based on chosen difficulty
- **Step 2:** Generate terrain using global difficulty
- **Step 3:** Boat drifts over time
- **Step 4:** When the boat drifts, check if the boat collides with a rock
- **Step 5:** When the boat collides with a rock, check if the boat health is 0.

Exceptions:

- **Step 1:** If difficulty is greater than the maximum, set difficulty to the maximum value.
- **Step 2:** If too many rocks are generated, set the number of rocks to the maximum possible value.
- **Step 3:** Boat will drift a different direction if drifting causes the boat to fall off the coordinate plane.

Post conditions: Player either dies or the boat is brought back to shore by the player.

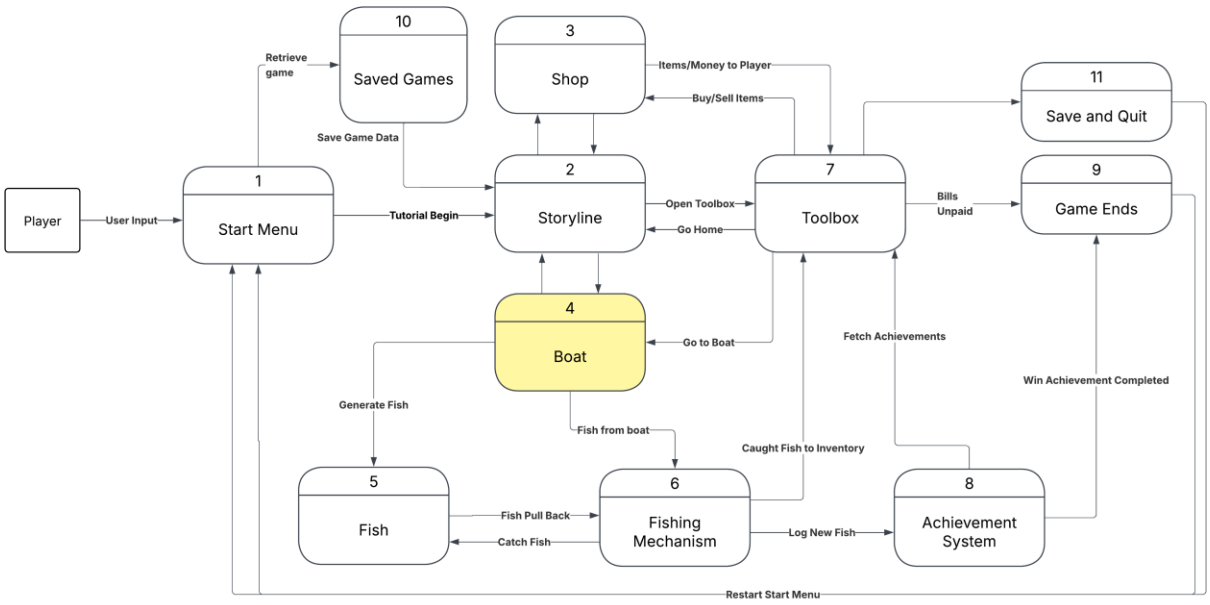
Priority: 1*

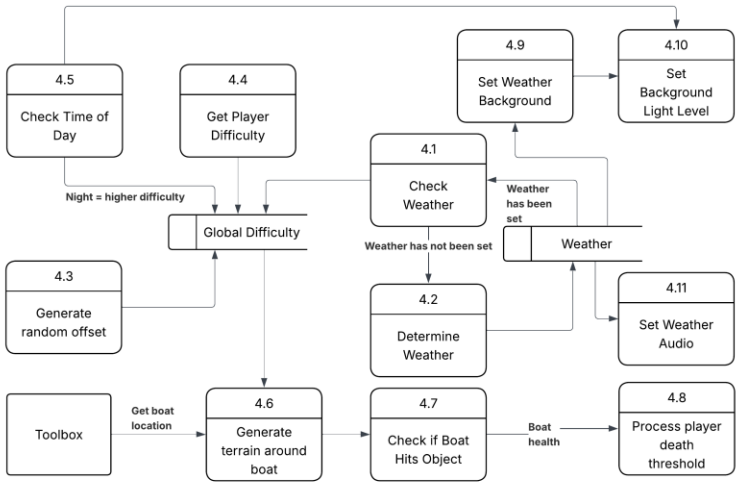
ID: BOAT

*The priorities are 1 = must have, 2 = essential, 3 = nice to have.

3. Data Flow diagram(s) from Level 0 to process description for your feature _____14

Data Flow Diagrams





Process Descriptions

```
include RandomNumberGenerator as RNG;

string DetermineWeather()
{
	LIST string possible_weather =
		{"clear", "rainy", "stormy", "cloudy"};

	float weatherValue = RNG.GenerateNumber(float, 0.00, 1.00);
	if (weatherValue < 0.10)
		return "stormy"
	else if (weatherValue < 0.30)
		return "rainy"
	else if (weatherValue < 0.60)
		return "cloudy"
	else
		return "clear"
}

void SetWeatherSigns(current_weather, light_level)
{
	Scene.SetBackground(current_weather, light_level);
	Scene.PlayAudio(current_weather);
}

float GenerateGlobalDifficulty(current_weather, time_of_day, player_difficulty)
{
	float global_difficulty = 1.0;
	float time_difficulty = 1.0;

	DICT weather_difficulties = {
		"stormy": 2.0
		"rainy": 1.5
		"cloudy": 1.0
		"clear": 0.75
	}

	DICT player_difficulties = {
		"casual": 1.0
		"hard": 2.0
		"drbc": 0.0
	};
```

```
	# In 24 hour time
	if (time_of_day >= 20 or time_of_day < 6)
		time_difficulty = 2.0;

	global_difficulty =
		weather_difficulties[current_weather] +
		player_difficulties[player_difficulty] +
		time_difficulty +
		RNG.GenerateNumber(float, 0.00, 0.25);

	return global_difficulty

}

TerrainObject GenerateTerrain(boat_position, global_difficulty)
{
	int min_rocks = 0;
	int max_rocks = global_difficulty * 10;
	int num_rocks = min_rocks;

	DICT locations = {
		north_sea: 2
		south_sea: 3
		shallow_sea: 1
		deep_sea: 4
	}

	int highest_num_rocks = num_rocks;
	for (locations[boat_position])
	{
		do:
			num_rocks = RNG.GenerateNumber(int, min_rocks, max_rocks);
		until:
			num_rocks > highest_num_rocks;

		highest_num_rocks = num_rocks;
	}

	ARRAY TerrainObject rocks[num_rocks];
	for (rock in rocks)
	{
```

```
		int new_rock_location

		do:
			new_rock_location = RNG.GenerateNumber(signed float, -20, 20);
		until:
			new_rock_location not in rock_locations;

		rock_locations[rock].rock_location = new_rock_location;
	}

	return rocks;
}

when boat.IsActive BECOMES true
{
	string current_weather = DetermineWeather();
	SetWeatherSigns(current_weather, day.game_clock.GetCurrentTime());

	float global_difficulty = GenerateGlobalDifficulty(current_weather,
day.game_clock.GetCurrentTime(), toolbox.difficulty);
	TerrainObject OceanRocks = GenerateTerrain(toolbox.boat_position, global_difficulty);
}

while boat.IsActive
{
	wait(15 minutes); # In-game time
	boat.position += RNG.GenerateNumber(signed float, -2, 2);
	if boat.position in OceanRocks
	{
		boat.Destroy();
		player.LoseGame();
	}
}
```

4. Acceptance Tests _____9

Several features are reliant on Random Number Generation (RNG). It's important to test the system to ensure the RNG is consistent:

Weather Determination: Weather values will be calculated 10,000 times. The expected spread should be:

- 1,000 stormy
- 2,000 rainy
- 3,000 cloudy
- 4,000 clear

Global Difficulty: Global difficulty is calculated using a randomly generated value. This value is expected to be 0.125 given the following expected value equation:

$$E[X] = \frac{0 + 0.25}{2} = \frac{0.25}{2} = 0.125$$

This value will be generated 10,000 times and will then be averaged. The result must be within one one-hundredth of the expected value.

Rock Locations: The locations of rocks will be generated based on global difficulty, but will be largely random. The boat lays on a two-dimensional Cartesian coordinate plane where -20 <= x <= 20 and -20 <= y <= 20. If the boat hits the rocks, the boat is destroyed.

There should be an even spread of rocks on the coordinate plane. A total of 100,000 rocks will be generated and the spread will be checked to ensure that it is roughly even.

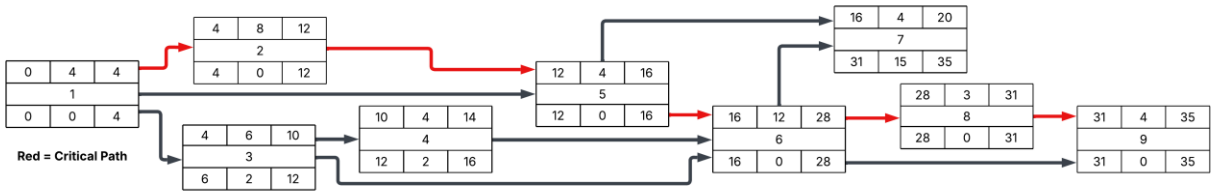
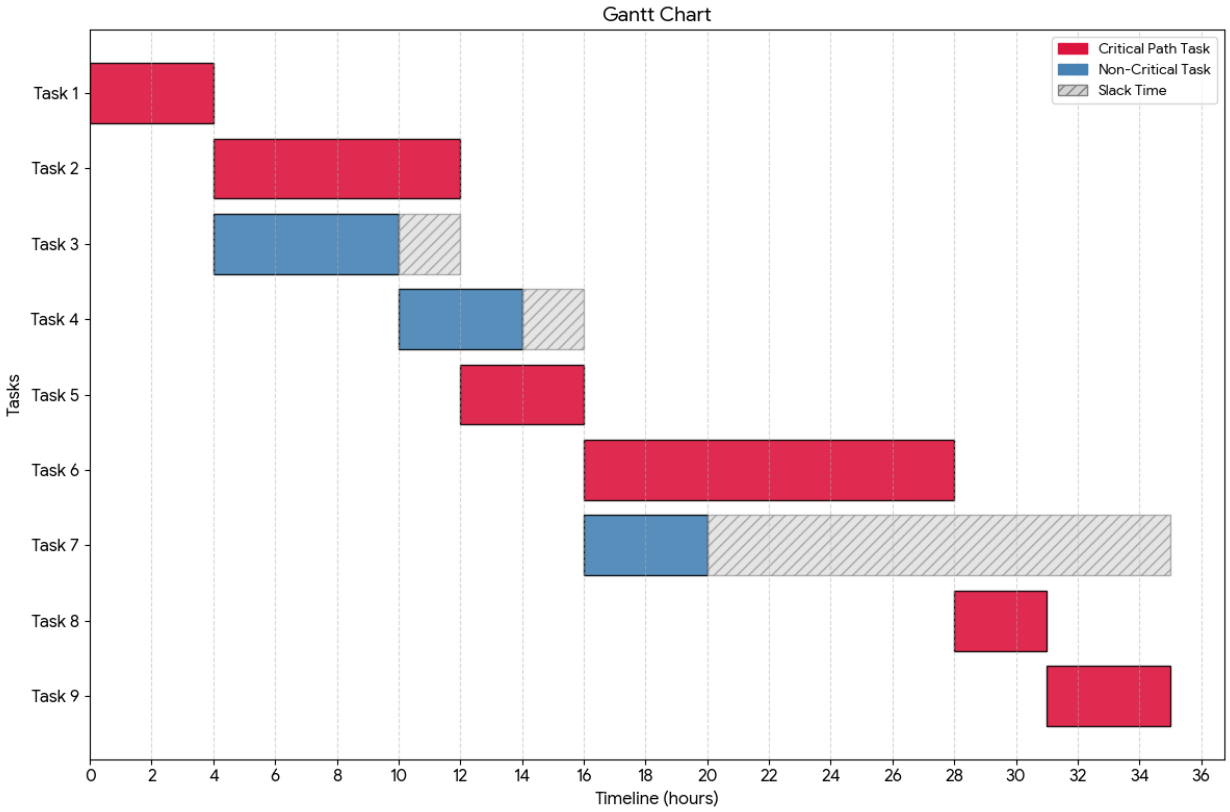
Boat Position: The position of the boat randomly changes (drifts) while the player is at sea. The amount by which it drifts must be checked to ensure fairness. Due to its signed nature, the value by which the boat drifts should average out to approximately 0. A total of 10,000 values will be generated and the average calculated.

5. Timeline _____/10

Work items

Task	Duration (Hours)	Predecessor Task(s)
1. Requirements Collection	4	-
2. Difficulty Offset Calculation	8	1
3. Weather Elements Design	6	1
4. Background Elements Design	4	3
5. RNG Programming	4	1, 2
6. Integration and Balancing	12	5, 3, 4
7. User Documentation	4	5
8. Testing	3	6
9. Installation	4	6, 8

Pert diagram



Gantt timeline